Chapter 16 | Exercise solutions with Python

From Pretrained Weights to Task Behavior

This notebook solves Exercises 5–9. It aligns response labels with logit rows, tests which perturbations change the loss, inspects a fresh low-rank adapter, and follows gradients through a classifier and a preference loss. Exercises 1–4 and 10–11 are in the written solutions.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements-ch05.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import numpy as np
import torch
from torch import nn
from torch.nn import functional as F

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch16_adaptation"
    )
    if model_path.is_dir():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

5. Check target alignment in code

Suppose the model has read `BOS Question Assistant` and is about to predict `New`. This prediction belongs to the logit row at `Assistant`. For an input row at position t, the next-token target comes from position $t + 1$. The mask first marks the **tokens to predict** in their original positions. The helper then shifts both the token IDs and the mask so that each target lines up with the preceding logit row.

Number the seven positions from zero. Use a long tensor with batch size one and mark positions 3, 4, and 5 as the response. The end marker `EOS` is part of the response because the model must learn when to stop. Padding at position 6 is excluded.

Logit row	Input token	Next token and ID	Surviving label
0	BOS	Question (3)	IGNORE
1	Question	Assistant (4)	IGNORE
2	Assistant	New (5)	5
3	New	Address (6)	6
4	Address	EOS (2)	2
5	EOS	PAD (0)	IGNORE

The helper uses `IGNORE = -100` as a label sentinel. It is outside the vocabulary and means this row contributes no supervised target. There is no target after the last supplied token, so the result has shape $[1, 6]$. The scoring helpers use the first six logit rows when given the original seven-row logits.

```
from examples.ch07_pretraining.data import IGNORE
from examples.ch16_adaptation.model import response_labels

target_names = ("BOS", "Question", "Assistant",
                "New", "Address", "EOS", "PAD")
target_ids = torch.tensor([[1, 3, 4, 5, 6, 2, 0]], dtype=torch.long)
target_mask = torch.tensor(
    [[False, False, False, True, True, True, False]],
    dtype=torch.bool,
)
target_labels = response_labels(target_ids, target_mask)
expected_labels = torch.tensor(
    [[IGNORE, IGNORE, 5, 6, 2, IGNORE]], dtype=torch.long
)
assert torch.equal(target_labels, expected_labels)
assert target_labels.shape == (1, 6)

print("IDs         ", target_ids.tolist())
print("Response    ", target_mask.tolist())
print("Shifted     ", target_ids[:, 1:].tolist())
print("Labels      ", target_labels.tolist())
for row, label in enumerate(target_labels[0].tolist()):
    if label != IGNORE:
        print(f"Row {row} ({target_names[row]}) scores "
              f"{target_names[row + 1]} (ID {label})")
```

OUTPUT

```
IDs          [[1, 3, 4, 5, 6, 2, 0]]
Response     [[False, False, False, True, True, True, False]]
Shifted      [[3, 4, 5, 6, 2, 0]]
Labels       [[-100, -100, 5, 6, 2, -100]]
Row 2 (Assistant) scores New (ID 5)
Row 3 (New) scores Address (ID 6)
Row 4 (Address) scores EOS (ID 2)
```

Notice that `Assistant` is a prompt token but its logit row is scored because the **next** token is `New`. Conversely, `EOS` is a response token but its own logit row is ignored because the next token is padding. Selecting targets and providing input context are separate operations.

6. Perturb a logit row

Keep the same labels. Give the model seven logit rows with seven vocabulary entries in each row. All entries start at zero, so softmax assigns probability $1/7$ to every token. Only the three targets at rows 2, 3, and 4 count. The mean loss is $\ln 7$, and the completion log-probability is the **sum** $-3 \ln 7$ for this one sequence.

First raise the `Question` score in row 0 (`BOS`) from zero to 2. This changes the row's probability of `Question`, but `Question` is an ignored target. The probability distribution at every other row stays the same.

For the second intervention, start again from all zeros and raise the `New` score in row 2 (`Assistant`) to 2. Row 2 now assigns `New` probability

$$p(\texttt{New}) = \frac{\exp(2)}{\exp(2) + 6}.$$

There are still six other vocabulary entries with score zero in that row. The negative log-probability of `New` becomes

$$d = -\ln p(\texttt{New}) = \ln(\exp(2) + 6) - 2 = \ln(1 + 6\exp(-2)).$$

Rows 3 and 4 still contribute $\ln 7$ each. Thus the new mean loss is $(d + 2\ln 7)/3$ and the new completion log-probability is $-d - 2\ln 7$. The denominator of the mean stays three because neither intervention changes the labels.

```
from examples.ch16_adaptation.model import (
    completion_logprob, response_loss,
)

score_zero = torch.zeros((1, 7, 7), dtype=torch.float64)
score_prompt = score_zero.clone()
score_prompt[0, 0, 3] = 2.0  # BOS row, Question column
score_response = score_zero.clone()
score_response[0, 2, 5] = 2.0  # Assistant row, New column

score_cases = (
    ("all zero", score_zero),
    ("boost Question at BOS", score_prompt),
    ("boost New at Assistant", score_response),
)
score_results = []
for name, scores in score_cases:
    mean_loss = response_loss(scores, target_labels)
    logprob = completion_logprob(scores, target_labels)
    score_results.append((mean_loss, logprob))
    print(f"{name:<24} loss {mean_loss.item():.9f}  "
          f"logp {logprob.item():.9f}")
```

```
OUTPUT
all zero                 loss 1.945910149  logp -5.837730447
boost Question at BOS    loss 1.945910149  logp -5.837730447
boost New at Assistant   loss 1.495419321  logp -4.486257962
```

Verify the two formulas independently. A mean loss and a summed log-probability have different units of aggregation. Here there is one sequence with exactly three scored positions, so multiplying its mean loss by -3 recovers the completion log-probability. This scaling would not turn a mean over a mixed batch into one sum per response.

```
score_uniform_nll = np.log(7.0)
score_new_nll = np.log1p(6.0 * np.exp(-2.0))
score_boosted_nll = (score_new_nll + 2 * score_uniform_nll) / 3
score_expected = (score_uniform_nll, score_uniform_nll,
                  score_boosted_nll)
for (mean_loss, logprob), expected in zip(score_results, score_expected):
    torch.testing.assert_close(
        mean_loss, score_zero.new_tensor(expected),
        rtol=0, atol=1e-12,
    )
    torch.testing.assert_close(
        logprob, score_zero.new_tensor([-3 * expected]),
        rtol=0, atol=1e-12,
    )

print(f"Boosted target probability: {np.exp(-score_new_nll):.9f}")
print(f"Boosted target NLL:         {score_new_nll:.9f}")
print("Both formulas agree within absolute tolerance 1e-12.")
```

```
OUTPUT
Boosted target probability: 0.551872816
Boosted target NLL:         0.594437664
Both formulas agree within absolute tolerance 1e-12.
```

Raising the correct score in a selected row makes the mean loss smaller and the summed log-probability less negative. The ignored-row experiment changes already-computed logits. During a model forward pass, an ignored prompt token can still influence later response logits through causal attention.

7. Inspect a fresh adapter

Imagine adding a second path to an existing four-input, three-output linear layer. The original path keeps its weight and bias. The new path first compresses the input from four features to two, then expands it to three. This is low-rank adaptation, or LoRA. A fresh adapter is built so this second path starts by adding zero.

Write the base weight as W_0, its bias as $\mathbf{b}_0$, and the two LoRA factors as A and B. The rank setting is $r = 2$ and the scale is $\alpha/r = 4/2 = 2$. Using column vectors, the computation is

$$\mathbf{y} = W_0\mathbf{x} + \mathbf{b}_0 + 2B(A\mathbf{x}).$$

The factor A has shape $[2, 4]$, so $A\mathbf{x}$ has two entries. The factor B has shape $[3, 2]$, so the update has three entries and can be added to the base output. The implementation initializes A randomly and B to zero. Consequently the update is the zero vector even if $A\mathbf{x}$ contains nonzero entries. The base bias still contributes to the output.

Create the input with `base.weight.new_tensor` so its dtype and device match the layer. This run uses CPU float64. The seed fixes the illustrative parameter values and the numeric output. The equality at initialization follows from $B = 0$ for any base initialization.

```
from examples.ch16_adaptation.model import LoRALinear

with torch.random.fork_rng(devices=[]):
    torch.manual_seed(1607)
    base = torch.nn.Linear(4, 3, bias=True, dtype=torch.float64)
    adapter = LoRALinear(base, rank=2, alpha=4)

lora_x = base.weight.new_tensor([1, 0, 0, 1])
with torch.no_grad():
    lora_base_output = base(lora_x)
    lora_output = adapter(lora_x)
    lora_update = adapter.scale * (
        adapter.b @ (adapter.a @ lora_x)
    )

assert adapter.scale == 2.0
assert torch.count_nonzero(adapter.b) == 0
torch.testing.assert_close(
    lora_update, torch.zeros_like(lora_update), rtol=0, atol=0
)
torch.testing.assert_close(
    lora_output, lora_base_output, rtol=0, atol=0
)

print("Adapter scale", adapter.scale)
print("Base output   ", [f"{v:.12f}" for v in lora_base_output])
print("Wrapped output", [f"{v:.12f}" for v in lora_output])
print("Adapter update", lora_update.tolist())
```

```
OUTPUT
Adapter scale 2.0
Base output    ['-0.006492101944', '0.270685236461', '-0.654326084645']
Wrapped output ['-0.006492101944', '0.270685236461', '-0.654326084645']
Adapter update [0.0, 0.0, 0.0]
```

Inspect all four parameter tensors. A `requires_grad` flag of `True` makes a parameter eligible to receive gradients. This wrapper sets the base flags to `False` and leaves both factors trainable.

Parameter	Shape	Number of scalars	Trainable
Base weight W_0	$[3, 4]$	12	No
Base bias $\mathbf{b}_0$	$[3]$	3	No
Factor A	$[2, 4]$	8	Yes
Factor B	$[3, 2]$	6	Yes

There are $2 \cdot 4 + 3 \cdot 2 = 14$ trainable scalars and $15 + 14 = 29$ scalars in the wrapper altogether. The zero-initialized factor still counts as trainable. Its stored values are zero now, while its gradient flag permits an optimizer to change them.

```
expected_lora = {
    "base.weight": ((3, 4), False),
    "base.bias": ((3,), False),
    "a": ((2, 4), True),
    "b": ((3, 2), True),
}
print(f"{'Parameter':<13} {'Shape':<8} {'Scalars':>7}  Grad flag")
for name, parameter in adapter.named_parameters():
    shape = tuple(parameter.shape)
    assert (shape, parameter.requires_grad) == expected_lora[name]
    print(
        f"{name:<13} {str(shape):<8} {parameter.numel():>7}  "
        f"{parameter.requires_grad}"
    )
lora_trainable = sum(
    p.numel() for p in adapter.parameters() if p.requires_grad
)
lora_total = sum(p.numel() for p in adapter.parameters())
assert lora_trainable == 14 and lora_total == 29
print(f"Trainable {lora_trainable} of {lora_total}")
```

```
OUTPUT
Parameter     Shape    Scalars  Grad flag
a             (2, 4)         8  True
b             (3, 2)         6  True
base.weight   (3, 4)        12  False
base.bias     (3,)            3  False
Trainable 14 of 29
```

8. Compare trainable weights

Train two versions of the same small classifier on the same eight examples. The frozen version lets only the task head change. The full version also lets the encoder learn. For each version, run one experiment with one optimizer step and a separate experiment with two. In particular, the two-step run starts over and does not resume from a trained model.

Save a snapshot of one untouched `TinyClassifier` and load it into four fresh instances. All four therefore start with the same encoder and the same zero head. Use only the first two outputs of `classification_data`, the training inputs and class labels. The helper trains with full-batch SGD, meaning one update uses all eight examples, and we give it the same learning rate of 0.2 in every run. Holding these choices fixed means that the encoder's trainable status and the number of updates are the only differences in the comparison.

```
from examples.ch16_adaptation.model import (
    TinyClassifier, train_classifier,
)
from examples.ch16_adaptation.run_examples import classification_data

train_x, train_y = classification_data()[:2]
classifier_start = {
    name: tensor.detach().clone()
    for name, tensor in TinyClassifier().state_dict().items()
}
classifier_lr = 0.2
classifier_runs = {}
for steps in (1, 2):
    for mode in ("frozen", "full"):
        fresh = TinyClassifier()
        fresh.load_state_dict(classifier_start)
        train_classifier(
            fresh, train_x, train_y,
            freeze_encoder=(mode == "frozen"),
            steps=steps, lr=classifier_lr,
        )
        classifier_runs[steps, mode] = fresh

print(f"Training features {tuple(train_x.shape)} {train_x.dtype}")
print("Training class counts", train_y.bincount().tolist())
```

```
OUTPUT
Training features (8, 2) torch.float64
Training class counts [4, 4]
```

For each weight and bias tensor, subtract its original value and take the largest absolute entry in that difference. A zero means the whole tensor is unchanged. Float64 comparisons use absolute tolerance 10^{-12} and zero relative tolerance. The results distinguish changes from numerical residue in the bias entries.

```
classifier_atol = 1e-12
classifier_deltas = {
    key: {
        name: (p.detach() - classifier_start[name]).abs().max().item()
        for name, p in model.named_parameters()
    }
    for key, model in classifier_runs.items()
}
print("Steps  Mode    Encoder W   Encoder b   Head W      Head b")
for (steps, mode), changes in classifier_deltas.items():
    values = "   ".join(f"{change:.3e}" for change in changes.values())
    print(f"{steps:>5}  {mode:<6}  {values}")

for (steps, mode), changes in classifier_deltas.items():
    if steps == 1 or mode == "frozen":
        assert changes["encoder.weight"] <= classifier_atol
    else:
        assert changes["encoder.weight"] > classifier_atol
    assert changes["encoder.bias"] <= classifier_atol
    assert changes["head.weight"] > classifier_atol
    assert changes["head.bias"] <= classifier_atol

for steps in (1, 2):
    frozen = classifier_runs[steps, "frozen"]
    full = classifier_runs[steps, "full"]
    assert frozen.encoder.weight.grad is None
    assert full.encoder.weight.grad is not None
    for frozen_p, full_p in zip(
        frozen.head.parameters(), full.head.parameters()
    ):
        torch.testing.assert_close(
            frozen_p, full_p, rtol=0, atol=classifier_atol
        )
assert torch.count_nonzero(
    classifier_runs[1, "full"].encoder.weight.grad
) == 0
```

OUTPUT

```
Steps  Mode    Encoder W   Encoder b   Head W      Head b
    1  frozen  0.000e+00   0.000e+00   8.237e-02   0.000e+00
    1  full    0.000e+00   0.000e+00   8.237e-02   0.000e+00
    2  frozen  0.000e+00   0.000e+00   1.568e-01   2.776e-18
    2  full    8.305e-03   3.469e-19   1.568e-01   2.776e-18
```

Why do both encoders stay put on step 1? Let $\mathbf{a}_i = E\mathbf{x}_i + \mathbf{c}$ be the encoder's pre-activation vector and $\mathbf{h}_i = \tanh(\mathbf{a}_i)$ its feature vector for example i. The head weight H and bias $\mathbf{b}$ give logits $\mathbf{z}_i = H\mathbf{h}_i + \mathbf{b}$. Let $\mathbf{g}_i$ be the cross-entropy gradient for example i with respect to its logits before batch averaging. The gradient components are the predicted probabilities minus the one-hot target.

At initialization, every logit is zero. The two predicted probabilities are both $1/2$, so the target class has gradient $-1/2$ and the other class has gradient $1/2$. These nonzero gradients reach the head weight, even though its entries are currently zero. For a batch of m inputs the weight gradient is an average of outer products,

$$\nabla_H L = \frac{1}{m} \sum_{i=1}^{m} \mathbf{g}_i \mathbf{h}_i^\top.$$

The encoder receives a different gradient. Backpropagation multiplies the head's logit gradient by the transpose of its weight, then applies the elementwise derivative of $\tanh$. With $\odot$ denoting elementwise multiplication, the gradient of the mean loss at the encoder pre-activation is

$$\nabla_{\mathbf{a}_i} L = \frac{1}{m}(H^\top \mathbf{g}_i) \odot (1 - \mathbf{h}_i^2).$$

At the first backward pass $H = 0$, so the encoder gradient vanishes even in the full run, where those weights have `requires_grad=True`. SGD then updates H. On the second backward pass the nonzero head weight passes a gradient to the encoder. The full encoder weight changes at that step, and the frozen encoder remains unchanged. The frozen weight has no gradient stored, while the trainable encoder's first gradient is an actual tensor of zeros.

Both head weights agree after one step and after two. At the start of step 2, the head and encoder still have the same values in both runs. The head's gradient is therefore the same in that backward pass, before the optimizer changes the full encoder. If training continues, later head updates can differ because the encoders now supply different features.

The first head-bias gradient sums to zero because the labels are balanced and the initial predictions are uniform. The tiny bias differences in the second-step runs are at the scale of floating-point rounding in this symmetric example. Comparing the weight tensors separately avoids mistaking those tiny entries for substantial changes.

Check the explanation directly by constructing the head update at step 1 and the encoder weight update at step 2 from the formulas above. The calculation below uses matrix products and derivatives explicitly, with no call to backward. Inputs are rows, so the matrix formulas use the corresponding transposes.

```
targets_one_hot = torch.nn.functional.one_hot(
    train_y, num_classes=2
).to(dtype=train_x.dtype)
encoder_w0 = classifier_start["encoder.weight"]
encoder_b0 = classifier_start["encoder.bias"]
head_w0 = classifier_start["head.weight"]
head_b0 = classifier_start["head.bias"]
hidden0 = (train_x @ encoder_w0.T + encoder_b0).tanh()
batch_size = train_x.shape[0]

logits0 = hidden0 @ head_w0.T + head_b0
residual0 = logits0.softmax(dim=-1) - targets_one_hot
head_grad0 = residual0.T @ hidden0 / batch_size
head_w1 = head_w0 - classifier_lr * head_grad0
head_b1 = head_b0 - classifier_lr * residual0.mean(dim=0)

logits1 = hidden0 @ head_w1.T + head_b1
residual1 = logits1.softmax(dim=-1) - targets_one_hot
preact_grad1 = (residual1 @ head_w1) * (1 - hidden0.square())
encoder_grad1 = preact_grad1.T @ train_x / batch_size
encoder_w2 = encoder_w0 - classifier_lr * encoder_grad1

torch.testing.assert_close(
    head_w1, classifier_runs[1, "full"].head.weight,
    rtol=0, atol=classifier_atol,
)
torch.testing.assert_close(
    encoder_w2, classifier_runs[2, "full"].encoder.weight,
    rtol=0, atol=classifier_atol,
)
expected_change = (encoder_w2 - encoder_w0).abs().max().item()
print(f"Manual step-2 encoder weight change {expected_change:.9f}")
```

OUTPUT

```
Manual step-2 encoder weight change 0.008305283
```

9. Keep the reference fixed (optional)

Direct Preference Optimization (DPO) compares the policy's preference for a chosen response over a rejected response with the preference already expressed by a reference model. The policy gap is $-2-(-3.5)=1.5$. The reference gap is $-3-(-3.25)=0.25$. With $\beta=1$ the margin is

$$m=\beta[(p_c-p_r)-(r_c-r_r)]=1.25.$$

Here p_c, p_r denote policy log-probabilities and r_c, r_r denote reference log-probabilities. For one pair the loss is $L=-\ln\sigma(m)=\ln(1+\exp(-m))$, where $\sigma(z)=1/(1+\exp(-z))$. Its derivative with respect to the margin is

$$\frac{dL}{dm}=-\frac{1}{1+\exp(m)}=-\sigma(-m).$$

The policy scores enter the margin with derivatives $+\beta$ for the chosen response and $-\beta$ for the rejected response. Their loss gradients are therefore $-\beta\sigma(-m)$ and $+\beta\sigma(-m)$. If the scores could be updated directly, gradient descent would raise the chosen score and lower the rejected one. In a model these gradients flow through the response log-probabilities to shared trainable weights.

In the code, the reference scores are detached before their difference is computed. They supply the value 0.25 for the forward pass, while autograd has no path from the loss back to their original tensors. Give all four tensors `requires_grad=True` so the test distinguishes detachment from simply starting with gradients disabled.

```
from examples.ch16_adaptation.model import dpo_loss

dpo_names = ("policy chosen", "policy rejected",
             "reference chosen", "reference rejected")
dpo_scores = tuple(
    torch.tensor(x, dtype=torch.float64, requires_grad=True)
    for x in (-2.0, -3.5, -3.0, -3.25)
)
dpo_value = dpo_loss(*dpo_scores, beta=1)
dpo_value.backward()

print(f"Loss: {dpo_value.item():.12f}")
for name, score in zip(dpo_names, dpo_scores):
    grad = "None" if score.grad is None else f"{score.grad.item():+.12f}"
    print(f"{name:<19} requires_grad={score.requires_grad}  grad={grad}")
```

```
OUTPUT
Loss: 0.251929081345
policy chosen       requires_grad=True  grad=-0.222700138825
policy rejected     requires_grad=True  grad=+0.222700138825
reference chosen    requires_grad=True  grad=None
reference rejected  requires_grad=True  grad=None
```

```
dpo_expected_margin = 1.5 - 0.25
dpo_expected_grad = 1.0 / (1.0 + np.exp(dpo_expected_margin))
torch.testing.assert_close(
    dpo_value,
    dpo_value.new_tensor(np.logaddexp(0.0, -dpo_expected_margin)),
    rtol=0, atol=1e-12,
)
for score, sign in zip(dpo_scores[:2], (-1, 1)):
    torch.testing.assert_close(
        score.grad, score.new_tensor(sign * dpo_expected_grad),
        rtol=0, atol=1e-12,
    )
assert all(score.requires_grad for score in dpo_scores)
assert all(score.grad is None for score in dpo_scores[2:])
print("Loss and policy gradients match the scalar derivation.")
print("Both reference gradients are None.")
```

OUTPUT

```
Loss and policy gradients match the scalar derivation.
Both reference gradients are None.
```

Both reference gradient fields stay `None`. In this fresh backward pass, `None` means no gradient was accumulated for that tensor. A gradient tensor containing zero would instead represent a computed value. Detachment affects this loss's graph without changing the original reference tensors' `requires_grad` flags.